

Work Actuals Tracking, Implementation Plan

Status: Plan only. No feature code, no migrations applied.

1. Findings

1.1 The load-bearing join (`replicated_work` ↔ `generated_work` alignment)

`replicated_work` carries: `name`, `status`, `estimate` (numeric), `source_asset_record_id`, `canonical_asset_id`, `source_record_id`, `source_hash`, `meta`. It carries **no `craft` field and no `workflow_step_id`**. It knows nothing about the ArtHound workflow model.

`generated_work` carries: `work_name`, `workflow_step_id`, `craft`, `estimate_days`, `variable_values`, `start_date`, `end_date`, `deleted_at`.

The write-back path (`routes/schedule.py` → `_write_back_to_source`) stores the created source record ID back into `generated_work.source_record_id` for both Jira and Airtable. This is the only correlation token between the plan side and the live work side. It is per-item, not a batch token.

Consequence for per-craft reconciliation: There is no shared discriminator between the two tables at the row level beyond `canonical_asset_id` + the recovered `source_record_id`. A join on (`canonical_asset_id`, `source_record_id`) works when write-back succeeded and the source record was not cloned or restructured. For work that was created outside ArtHound, or where the source record ID was not stamped (write-back failure, pre-write-back data), there is no direct key, only fuzzy matching (name similarity, date overlap, estimate proximity) is available. This drives the m:n reconciliation design.

1.2 Sync strategy for `replicated_work`

`writer.py` upserts on (`owner_type`, `owner_id`, `source_type`, `source_record_id`), the natural key. It is **in-place upsert, not truncate-and-rebuild**. Orphan deletion runs only on full sync and only for IDs not present in the fetched set.

Implication: `replicated_work` rows have stable physical identity as long as the source record ID does not change. A `canonical_work` layer is still necessary, not because rows churn, but because: - `canonical_work` must survive source migrations (Jira → ShotGrid) where source record IDs will differ - The changelog and reconciliation tables are durable ArtHound-owned satellites; FKing them directly to `replicated_work` would break on any source credential reset, orphan deletion, or full-resync that drops rows - The upsert-stable guarantee is contingent on no source migration; `canonical_work` provides the migration-proof layer

`canonical_work` is therefore necessary but the upsert-stable property means the canonicalization path is lower urgency: row churn will not silently break the changelog in normal operations.

1.3 The differ

`lib/sync/differ.py` is intentionally thin: it compares `source_hash` (SHA-256 of `record.fields` JSON) and returns the records-to-upsert list. **Zero field-level diffing; no delta events are emitted.** The differ has no output channel for field-level transitions, it only answers "has anything changed?" The changelog must be built on top of it by:

1. Querying the current row's values before the upsert
2. Diffing old vs new at the field level after a hash change is detected
3. Persisting the deltas

This requires a new hook in the sync pipeline, not a change to `differ.py` itself. `differ.py` should remain a pure function; the field-delta logic lives in the runner or a new `changelog.py` module.

1.4 Effort signal availability

No worklog or time-tracking fields are mapped anywhere in the current connectors or normalizer. The normalizer's `_SOURCE_NATIVE_NAME_PATTERNS` explicitly suppresses `timespent`, `timeestimate`, `timeoriginalestimate`, `workratio`, and the aggregate variants. The Jira adapter type map lists `timetracking` as "computed" (meaning it arrives as a compound object and is not further deserialized). `routes/assets.py` adds the display-name variants ("Time Spent" , "Remaining Estimate" , etc.) to `_META_HIDDEN` so they are stripped before the frontend.

The suppression is intentional: Jira's native time-tracking is unreliable (aggregated, not per-craft, easy to leave blank) and Airtable has no native worklog concept.

Consequence: Active-effort and logged-effort razor types are only achievable if studios expose a studio-defined numeric field (e.g. an Airtable "Actual Days" or "Logged Hours" field, or a Jira custom field) and map it through the razor config. The built-in Jira worklog API (`/rest/api/3/issue/{id}/worklog`) requires a separate per-issue HTTP call and is not currently part of the sync pipeline. Elapsed-time razors (`end_date` - `start_date`, or `synced_at` - `created_at`) are achievable immediately from data already in the system. **Tier 0 should offer elapsed-time razors only; active/logged razors require Tier 1 plus explicit studio field mapping.**

1.5 Write-back correlation token

When `_write_back_to_source` succeeds, it stores the created source record ID back into `generated_work.source_record_id`. This is the ArtHound-controlled correlation token. It is: - Per-item (not per-batch) - Stamped only after successful source-tool creation - Not stamped if write-back fails (partial failure is silently swallowed) - The reverse: the next sync of that source record will land in `replicated_work` with the same `source_record_id`

This is a genuine correlation bridge. The join `generated_work.source_record_id = replicated_work.source_record_id` (within the same studio scope) is the high-confidence match method. Reconciliation should treat this as `match_method = 'source_id'`, `confidence = 1.0`. All other methods (name, date, estimate proximity) are fuzzy and carry lower confidence. No additional

correlation token is needed, but the reconciliation system must handle the case where `source_record_id` on `generated_work` is null (write-back not attempted or failed).

1.6 source_field_mappings structure

One row per `(owner_type, owner_id, source_type)`. The `mappings` column is a JSONB array of objects:

```
{
  "source_field_id":      "fldXXX",
  "source_field_name":    "Actual Days",
  "source_field_type":    "number",
  "source_field_options": {...},
  "arthound_slot":       null,
  "meta_bucket":         "custom",
  "display_tier":        "secondary",
  "ingest_suppressed":    false
}
```

The razor config must reuse this field catalog. A studio selecting an "actuals" field should pick from the fields already classified in their `source_field_mappings`. The razor config is a separate table that references `(owner_type, owner_id, source_type, source_field_id)`, not embedded in `source_field_mappings`, because it carries additional metadata (razor type, unit, conversion factor) that does not belong in the field catalog.

1.7 Consumers of work-level data

- `routes/numbersbot.py`: reads `replicated_work` (name, status, estimate) and `generated_work` (work_name, craft, estimate_days, start_date, end_date). Consumes both tables in context-building. Adding an actuals column to `replicated_work` and summary rollup fields to a future route would slot naturally into the existing parallel-fetch pattern.
- `routes/schedule.py`: writes `generated_work`, does source write-back, has `/reconcile-work` endpoint (soft-deletes orphaned snapshots). The existing reconcile logic does a set-difference on `source_record_id`, it does not update any actuals. The reconciliation table (plan-vs-actual) is a new concept not present here.
- `routes/reviews.py`: does not read `replicated_work` or `generated_work`. Not affected.
- `routes/assets.py`: does not read `replicated_work` directly. Not affected.
- A canonical_work layer would be a new join target for the above, no existing route queries a canonical_work table. The impact is additive, not breaking.

1.8 canonical_assets shape reference

After `20260509000003_schema_renames.sql`, `canonical_assets` is:

```
id                uuid PK
studio_id         uuid NOT NULL FK studios(id)
source_record_id  text NOT NULL
source_type       text NOT NULL
created_at        timestampz
UNIQUE (studio_id, source_record_id, source_type)
```

The `studio_id` (not `owner_type / owner_id`) is used here because canonical identity is always studio-owned, vendors access canonical assets via dispatches, never by direct ownership. `canonical_work` should mirror this: keyed on `(studio_id, source_record_id, source_type)` with `studio_id` not `owner_type/owner_id`, since work tracking is a studio-side concern and vendors access it (if at all) through the dispatch/payload model.

1.9 canonical_products dependency

No genuine dependency found. The actuals system links `canonical_work` → `canonical_asset` → `studio` and that chain is sufficient. Products appear in `replicated_assets.product` (a denormalized string) and in `replicated_products`, but neither the changelog, the reconciliation table, nor the actuals razor requires a stable product identity. Tier 0 rollups are by craft (from `generated_work.craft`) or by canonical asset. Product-level rollups can be computed as `SUM` group by `replicated_assets.product`, no `canonical_products` needed. **No canonical_products required. Constraint holds.**

2. Key Decisions

2.1 canonical_work: yes, deferred to Tier 1

Recommendation: Build `canonical_work` in Tier 1, not Tier 0. Tier 0 reads are aggregate rollups from `generated_work` and `replicated_work` directly, no durable satellite is written, so `canonical_work` is not needed. Tier 1 writes the changelog and reconciliation rows, which require the stable FK anchor.

Shape: Mirror `canonical_assets` exactly:

```
id                uuid PK
studio_id         uuid NOT NULL FK studios(id)
source_record_id  text NOT NULL
source_type       text NOT NULL
created_at        timestampz
UNIQUE (studio_id, source_record_id, source_type)
```

Do not add `owner_type/owner_id`, canonical identity is studio-scoped by definition.

Trade-off: Delaying `canonical_work` means Tier 0 actuals are unlinked snapshots (no history).

Acceptable: Tier 0 is explicitly "baseline rollup / final-state parsing only." The moment a studio opts into Tier 1, the canonicalization backfill run establishes stable IDs for existing `replicated_work` rows.

Deciding question: Does any Tier 0 feature write a satellite that must survive source migration? No, Tier 0 is read-only. Decision is safe.

2.2 Changelog shape

Recommendation: Append-only event table keyed to `canonical_work_id`:

```
id                uuid PK
canonical_work_id uuid NOT NULL FK canonical_work(id)
sync_id           uuid (FK sync_log.id, nullable – allows manual imports)
observed_at       timestampz NOT NULL
field_name        text NOT NULL
old_value         text
new_value         text
```

Field values stored as text (coerced from any source type). Structured values (JSON arrays, select objects) serialized to canonical JSON string before storing, consistent comparison, no type ambiguity.

Trade-off: Text storage of all values is lossy for numeric precision. Alternative: JSONB

`old_value` / `new_value`. JSONB is better if downstream queries need to compare numeric deltas (estimate drift). Prefer JSONB.

Forward-instrument only: No pre-onboarding backfill. First sync after Tier 1 activation writes the initial snapshot as a single `(field, null, new_value)` event per field, not a state diff. This is a known limitation: first-seen is not first-changed.

Index: `(canonical_work_id, observed_at DESC)`, covers "show me the history of this work item" queries.

2.3 Reconciliation m:n model

Recommendation: Junction table between `generated_work` (plan side) and `canonical_work` (actual side):

```
id                uuid PK
generated_work_id  uuid FK generated_work(id)      -- nullable: historical actuals with no
canonical_work_id  uuid NOT NULL FK canonical_work(id)
match_method       text NOT NULL  -- 'source_id' | 'name_fuzzy' | 'manual' | 'unmatched'
confidence         numeric(4,3)  -- 0.000–1.000
contribution_ratio numeric(5,4)  -- 0.0000–1.0000; plan share when a plan maps to multiple
reconciled_at      timestampz NOT NULL DEFAULT now()
notes             text           -- for manual matches
```

`generated_work_id` is nullable to allow historical actuals (work that completed before ArtHound was adopted, or work created directly in the source without a generated plan).

Trade-off: Allowing null `generated_work_id` means the table is not a true m:n bridge for every row. Alternative: a sentinel `generated_work` row for "no plan." Null is cleaner, the "no plan side" state is an intentional domain concept, not an error.

contribution_ratio: Required for split work (one plan item matched to N actual items, each representing a portion of the estimate). The sum of `contribution_ratio` for all actuals linked to one `generated_work_id` must equal 1.0 (enforced by application logic, not DB constraint, because it spans rows).

2.4 Razor type system

Recommendation: Typed enum with coercion to a canonical actual-unit set. No free-form config.

Canonical actual types: - `elapsed_total`, wall-clock time from work item creation to close (or `synced_at` of terminal status). Available immediately from existing data if status transition is captured. No studio field required. - `elapsed_active`, wall-clock time from first "in-progress" status to first "done" status. Requires changelog (Tier 1) to detect the transition timestamps. - `logged_effort`, sum of a studio-defined numeric field (e.g. "Logged Days", "Actual Hours"). Requires studio to configure the field and declare the unit. - `active_effort`, same as `logged_effort` but semantically represents effort excluding wait/blocked time (studio declares the distinction). - `declared_custom`, studio-defined numeric field with explicit label and unit. Functionally identical to `logged_effort` but allows studios to name it meaningfully (e.g. "render frames" for technical work).

The razor config row declares which type is active, which source field drives it (for logged/active/declared types), and the declared unit (days, hours, frames, or a studio-defined string).

Reuse of normalizer/alias machinery: The razor config references a `source_field_id` from the studio's `source_field_mappings`. The normalizer already classifies and names these fields. Razor config lookup is `WHERE owner_type = $1 AND owner_id = $2 AND source_type = $3`, same pattern as field mappings, different table.

2.5 Commensurability guardrail

Non-negotiable. Variance delta (`actual - estimate`) is only computed when: 1. `estimate` is in `estimate_days` (the generated_work unit, days) AND 2. The razor's declared unit is also days, OR 3. An explicit `conversion_to_days` factor is stored in the razor config row

If condition 2 or 3 is not met, the variance API endpoint returns `null` with `commensurability: false` and a human-readable `unit_mismatch` message. It never silently returns a number with mixed units.

Placement: Enforced at the read path (the API route that computes variance), not at write time. The actuals value is stored as-is with its declared unit; commensurability is checked at query time when a comparison is requested. This allows the razor type to change (e.g. studio switches from frames to days) without corrupting historical actuals.

3. Schema

3.1 Migration sequence overview

All migrations follow expand/contract with backfill-before-drop. Column fallbacks retained until production verification. Irreversible steps flagged.

```

Tier 0:
M-01 actuals_razor_config (new table - reversible)
M-02 replicated_work.actual_value (additive column - reversible)

Tier 1:
M-03 canonical_work (new table - reversible)
M-04 replicated_work.canonical_work_id FK (additive column - reversible)
M-05 canonical_work backfill (data migration - reversible pre-Tier-1 activation)
M-06 work_changelog (new table - reversible)
M-07 work_reconciliation (new table - reversible)
M-08 replicated_work.canonical_work_id NOT NULL (constraint tighten - IRREVERSIBLE post-

```

Column drops follow only after production verification queries return 0 anomalies (same gate as the `_slots` demotion pattern). Steps M-03 through M-07 are independently reversible because they are additive. M-08 is the first irreversible step and should only run after Tier 1 is verified in production.

3.2 M-01: actuals_razor_config

```

create table actuals_razor_config (
  id                uuid primary key default gen_random_uuid(),
  owner_type        text not null check (owner_type in ('studio', 'vendor')),
  owner_id          uuid not null,
  source_type       text not null,
  razor_type        text not null check (razor_type in (
    'elapsed_total', 'elapsed_active',
    'logged_effort', 'active_effort', 'declared_custom'
  )),
  -- For logged/active/declared types: which field in source_field_mappings drives the value
  source_field_id   text,
  source_field_name text, -- display name; denormalized for read efficiency
  -- Declared unit of the actual value.
  declared_unit     text not null default 'days'
    check (declared_unit in ('days', 'hours', 'frames', 'custom')),
  custom_unit_label text, -- populated only when declared_unit = 'custom'
  -- Conversion factor to days (for commensurability with estimate_days).
  -- NULL means "no conversion defined" - variance computation refused.
  conversion_to_days numeric(10,6),
  enabled           boolean not null default true,
  created_at        timestampz default now(),
  updated_at        timestampz default now(),

  unique (owner_type, owner_id, source_type)
);

create index on actuals_razor_config (owner_type, owner_id);
alter table actuals_razor_config enable row level security;

create policy "arc_studio" on actuals_razor_config for all using (
  owner_type = 'studio' and
  owner_id = (auth.jwt() -> 'app_metadata' -> 'studio_id')::uuid
);
create policy "arc_vendor" on actuals_razor_config for all using (
  owner_type = 'vendor' and
  owner_id = (auth.jwt() -> 'app_metadata' -> 'vendor_id')::uuid
);

```

3.3 M-02: replicated_work.actual_value (additive, Tier 0)

```
alter table replicated_work
  add column if not exists actual_value numeric;
-- populated by the sync pipeline when the studio's razor config maps a source field
-- to this column. NULL = not configured or source field absent.
-- unit declared in actuals_razor_config.declared_unit - not stored per-row.
```

No backfill needed, starts NULL. No column fallback required. Safe to drop if Tier 0 is rolled back (no downstream FK).

3.4 M-03: canonical_work (Tier 1)

```
create table canonical_work (
  id                uuid primary key default gen_random_uuid(),
  studio_id         uuid not null references studios(id),
  source_record_id  text not null,
  source_type       text not null,
  created_at        timestampz default now(),

  unique (studio_id, source_record_id, source_type)
);

create index on canonical_work (source_record_id);
alter table canonical_work enable row level security;

create policy "cw_studio" on canonical_work for all using (
  studio_id = (auth.jwt()→'app_metadata'→>'studio_id')::uuid
);
```

No RLS for vendor read, vendors access via dispatch model, not direct table access. If vendor actuals tracking is required in a future iteration, add a separate join through `payload_dispatches`.

3.5 M-04: replicated_work.canonical_work_id (expand phase, Tier 1)

```
alter table replicated_work
  add column if not exists canonical_work_id uuid references canonical_work(id);

create index on replicated_work (canonical_work_id);
```

Column starts NULL. Backfill happens in M-05. The NOT NULL constraint is added in M-08 after verification.

3.6 M-05: canonical_work backfill (Tier 1 activation gate)

Backfill run as a one-time script (not a migration file), executed per-studio when they opt into Tier 1:

```

-- Insert canonical_work rows for all replicated_work rows without one.
-- Idempotent: ON CONFLICT DO NOTHING.
INSERT INTO canonical_work (studio_id, source_record_id, source_type)
SELECT DISTINCT
    owner_id::uuid,
    source_record_id,
    source_type
FROM replicated_work
WHERE owner_type = 'studio'
    AND owner_id = $studio_id
    AND canonical_work_id IS NULL
ON CONFLICT (studio_id, source_record_id, source_type) DO NOTHING;

-- Back-fill the FK on replicated_work.
UPDATE replicated_work rw
SET canonical_work_id = cw.id
FROM canonical_work cw
WHERE rw.owner_type = 'studio'
    AND rw.owner_id = cw.studio_id::text::uuid
    AND rw.source_record_id = cw.source_record_id
    AND rw.source_type = cw.source_type
    AND rw.canonical_work_id IS NULL;

```

Verification gate (must return 0 before M-08):

```

SELECT count(*) FROM replicated_work
WHERE owner_type = 'studio'
    AND owner_id = $studio_id
    AND canonical_work_id IS NULL;

```

3.7 M-06: work_changelog (Tier 1)

```

create table work_changelog (
    id                uuid primary key default gen_random_uuid(),
    canonical_work_id uuid not null references canonical_work(id) on delete cascade,
    sync_id           uuid references sync_log(id),
    observed_at       timestampz not null default now(),
    field_name        text not null,
    old_value          jsonb,
    new_value          jsonb
);

create index on work_changelog (canonical_work_id, observed_at desc);
create index on work_changelog (sync_id);
alter table work_changelog enable row level security;

-- RLS: read access via canonical_work → studio_id
create policy "wc_studio" on work_changelog for select using (
    canonical_work_id in (
        select id from canonical_work
        where studio_id = (auth.jwt()→'app_metadata'→>'studio_id')::uuid
    )
);

-- Insert is service-role only (sync pipeline writes via service role key).

```

Retention: no automatic trim initially. Add a nightly retention policy (similar to sync_log) once volume is observed, estimated at ~20 change events per active work item per quarter.

3.8 M-07: work_reconciliation (Tier 1)

```
create table work_reconciliation (
  id                uuid primary key default gen_random_uuid(),
  generated_work_id  uuid references generated_work(id), -- nullable: actuals with no plan
  canonical_work_id  uuid not null references canonical_work(id),
  match_method       text not null
                    check (match_method in ('source_id', 'name_fuzzy', 'manual', 'unmatched')),
  confidence         numeric(4,3) not null check (confidence between 0 and 1),
  contribution_ratio numeric(5,4) not null default 1.0
                    check (contribution_ratio between 0 and 1),
  reconciled_at      timestampz not null default now(),
  notes              text
);

create index on work_reconciliation (generated_work_id);
create index on work_reconciliation (canonical_work_id);
-- Unique actual-plan pair per reconciliation pass
create unique index on work_reconciliation (generated_work_id, canonical_work_id)
  where generated_work_id is not null;

alter table work_reconciliation enable row level security;
create policy "wr_studio" on work_reconciliation for select using (
  canonical_work_id in (
    select id from canonical_work
    where studio_id = (auth.jwt() -> 'app_metadata' -> 'studio_id')::uuid
  )
);
```

3.9 M-08: NOT NULL constraint on replicated_work.canonical_work_id (IRREVERSIBLE)

Run only after M-05 verification gate passes in production:

```
-- IRREVERSIBLE - only run after verification query returns 0 for all studios
alter table replicated_work
  alter column canonical_work_id set not null;
```

4. Code Changes

4.1 lib/sync/normalizer.py

Tier 0, actual_value extraction: - Add `_WORK_ACTUAL_ALIASES` list (similar to `_WORK_ESTIMATE_ALIASES`) as a fallback for studios without razor config. - Add optional `razor_config: dict | None` parameter to `normalize_work()`. - When `razor_config` is provided and has `source_field_id`, extract and coerce the value from `record.fields` into the returned dict as `"actual_value"`. - When `razor_config` is absent, `actual_value` is omitted from the output.

Tier 1, no normalizer changes needed. Changelog diffing operates on the serialized row, not on the raw record.

4.2 lib/sync/writer.py

Tier 0: - Add `actual_value` to the `upsert_work` row dict when present in the normalized record.

Tier 1: - Add `canonical_work_id` to the `upsert_work` row dict. - Add `upsert_canonical_work(studio_id, source_type, records)` function: batch-upserts `canonical_work` rows and returns `{source_record_id: canonical_work_id}` map (same pattern as `get_or_create_canonical_ids` in `lib/canonical.py`). - No changes to `load_existing_hashes`, it already operates on `replicated_work` columns.

4.3 New: lib/sync/changelog.py (Tier 1)

Single module for field-delta capture. Called from the runner after the upsert phase.

```
async def capture_work_deltas(
    owner_type: str,
    owner_id: str,
    source_type: str,
    upserted_records: list[dict], # normalized records that passed the hash filter
    canonical_work_map: dict[str, str], # source_record_id → canonical_work_id
    sync_id: str,
    tracked_fields: set[str] | None = None, # if None, track all non-meta fields
) → int: # returns count of delta events written
```

Implementation sketch: 1. For each upserted record that has a `canonical_work_id`, fetch the current row from `replicated_work` (before the upsert). This requires a pre-fetch step. 2. Compare old vs new for tracked fields. Tracked fields for work: `name`, `status`, `estimate`, `actual_value`, plus any field in `meta` that the studio has configured as tracked (future iteration). 3. For fields that changed, insert rows into `work_changelog`.

Pre-fetch ordering issue: The differ currently returns records-to-upsert but does not load the current field values. The runner must fetch current field values *before* calling `upsert_work`, limited to the records that passed the hash filter (not the full table). This is an $O(\text{upserted_batch})$ fetch, not $O(\text{all_work})$. Acceptable at current scales; add pagination if work tables grow beyond 10k rows per studio.

4.4 lib/sync/runner.py

Tier 0: - Load `actuals_razor_config` for the owner at sync start (one query, cached for the sync run). - Pass `razor_config` to `normalize_work()` calls. - Preserve the existing work sync flow.

Tier 1: - After fetching asset records and before normalizing work, call `upsert_canonical_work()` and capture the canonical map. - Pass `canonical_work_map` to `upsert_work`. - After `upsert_work`, call `capture_work_deltas()` with the upserted records and canonical map.

4.5 New: lib/sync/reconciliation.py (Tier 1)

Module to run the reconciliation pass. Called explicitly (not on every sync, reconciliation is an async background job, not a sync-blocking step).

Logic: 1. Fetch active `generated_work` rows with a `source_record_id` for the studio. 2. Fetch `canonical_work` rows whose `source_record_id` matches (high-confidence path, `match_method = 'source_id'`). 3. For unmatched `generated_work` rows: run name-fuzzy matching against `canonical_work` within the same `canonical_asset_id` (medium confidence, `match_method = 'name_fuzzy'`). 4. Upsert into `work_reconciliation`, preserving manual matches (`match_method = 'manual'`).

4.6 New: routes/actuals.py (Tier 0 read path)

New router at `/api/actuals`:

- `GET /rollup?studioId=&product=&craft=`, aggregate rollup: sum of `generated_work.estimate_days` vs sum of `replicated_work.actual_value`, grouped by craft. No `canonical_work` required. Returns commensurability status per group.
- `GET /asset/{canonical_asset_id}`, per-asset actuals summary. Tier 0: flat comparison of estimate sum vs `actual_value` sum per craft. Tier 1: reconciliation-linked comparison with confidence and `match_method` exposed.

Commensurability check is applied in every response that computes variance. If `razor_config` is absent or `conversion_to_days` is null and `declared_unit` $\neq$ 'days', the variance field is `null` and `unit_mismatch` is `true`.

4.7 routes/schedule.py

- `/reconcile-work` endpoint: extend to also invoke `lib/sync/reconciliation.py` when Tier 1 is active for the studio. The existing orphan-detection logic stays unchanged.

4.8 routes/numbersbot.py

- Extend `_parallel_fetch` to include a query for `actuals_razor_config` when `owner_type == 'studio'`.
- Extend the context block to include actuals rollup data (estimate vs actual by craft) when a razor is configured.
- Note: commensurability check applies here too, do not synthesize a numeric variance unless units are confirmed compatible.

4.9 Unchanged

- `lib/sync/differ.py`, no changes; remains a pure source-hash comparison
 - `lib/sync/connectors/airtable.py`, `lib/sync/connectors/jira.py`, no connector-level changes for Tier 0 or Tier 1; the `actual_value` field is just another numeric in the existing sync pipeline
 - `routes/assets.py`, `routes/reviews.py`, not affected
 - `lib/canonical.py`, not modified; `canonical_work` uses the same upsert-and-return pattern but implemented separately in `writer.py` to keep the canonical module asset-only
-

5. Tier 0 vs Tier 1 Split

Tier 0, Independently shippable

What it includes: - M-01: `actuals_razor_config` table - M-02: `replicated_work.actual_value` column - Razor config API: read/write for studio to declare their actuals field and unit - Normalizer change: extract `actual_value` when razor is configured - Writer change: write `actual_value` to `replicated_work` - `routes/actuals.py`, `/rollup` endpoint; per-asset summary (final-state reading only) - Commensurability guardrail in the read path - NumberBot context extension (actuals summary, units-checked)

What it does not include: `canonical_work`, changelog, reconciliation, any durable satellites, any history tracking.

What Tier 0 delivers: A studio can declare "actual days = field X" and immediately see aggregate plan-vs-actual rollups by craft, as long as their source tool carries a numeric actuals field. No onboarding beyond mapping one field.

Historical data in Tier 0: The final-state reading is correct for closed work items, `replicated_work.actual_value` at the time of query reflects the source tool's current value. If a studio has been logging actuals in Airtable for a year, those values appear immediately after the next sync. No reconstruction needed.

Tier 1, Opt-in, gates per studio

What it includes, in dependency order: 1. M-03: `canonical_work` table 2. M-04: `replicated_work.canonical_work_id` (expand phase) 3. M-05: Backfill script (run per-studio on opt-in) 4. M-06: `work_changelog` table 5. M-07: `work_reconciliation` table 6. `lib/sync/changelog.py`, field-delta capture in sync pipeline 7. `lib/sync/reconciliation.py`, reconciliation pass 8. M-08: NOT NULL constraint (IRREVERSIBLE, after verification) 9. Actuals route extension: reconciliation-linked per-asset detail 10. UI (deferred, not in this plan scope)

Tier 1 activation gate per studio: - Studio has had at least one full sync with Tier 0 (razor configured, `actual_value` populated) - M-05 backfill returns 0 nulls for the studio - Studio explicitly opts in via a settings action (prevents accidental activation)

Tier 1 delivers: Live flywheel, estimate vs actual tracked through time, transitions captured, plan-vs-actual reconciled by `source_id` match (high confidence) and name-fuzzy (medium confidence). Enables cycle-time analysis (`elapsed_active` razor requires changelog to detect status transitions).

6. Open Risks

6.1 Sync-interval aliasing

The sync cursor is a timestamp; delta syncs fetch records modified after the last sync. If a work item is created, updated, and set to a terminal status within a single sync interval, the changelog will only see the final state, not the intermediate transitions. The probability is low for multi-day work but non-zero for short-turnaround items (bug fixes, quick tasks). The charter is "forward instrument only", this is a known and accepted limitation. Document it in the UI as "transitions observed since ArtHound was activated."

6.2 Within-interval reversals

Related to 6.1: a status that advances and then reverts within one sync interval is invisible. The changelog will record the net change (or no change if it ended back at the prior value, the hash would match). The commensurability guardrail does not help here; this is a fundamental limitation of polling-based CDC. Mitigations: (a) webhook-triggered incremental syncs reduce the interval; (b) accept the limitation and document it.

6.3 Split/merge/disappearance handling

Split: One planned work item becomes two in the source tool. The `canonical_work` for the original item persists. The new item gets its own `canonical_work`. The reconciliation table links both new items to the original `generated_work_id` with `contribution_ratio = 0.5` each (or manually adjusted). Application logic must prevent `SUM(contribution_ratio) > 1.0` but cannot enforce this in a single-row DB constraint.

Merge: Two source records become one. One `canonical_work` will be linked in the reconciliation table; the other will have no further synced updates (eventual orphan). The orphan-detection logic in `/reconcile-work` (soft-delete of `generated_work` snapshots) should be extended to flag `canonical_work` rows whose `source_record_id` no longer exists in `replicated_work`.

Disappearance: A work item is deleted from the source tool. `delete_orphaned_records` in `writer.py` removes the `replicated_work` row on the next full sync. `canonical_work` is not deleted (it carries the durable history). The FK from `replicated_work.canonical_work_id` to `canonical_work(id)` uses no cascade, `canonical_work` rows survive replica deletion by design. The changelog and reconciliation rows survive via their own FK to `canonical_work`.

Disappearance policy decision (open): Should `canonical_work` rows be soft-deleted when the source record disappears? Recommend: add a `deleted_at` column to `canonical_work` (defaulting NULL, populated when source record is no longer present) mirroring the `generated_work` pattern. Exact policy, (defaulting NULL, populated when source record is no longer present) mirroring the `generated_work` pattern. Exact policy, how long to retain, how to surface in UI, is deferred.

6.4 Historical vs native divergence

Studios running before Tier 0 activation have existing `replicated_work` rows with no `actual_value`. After activation, future syncs populate it. This creates a split dataset: historical rows show NULL actuals, post-activation rows show values. Rollups that aggregate across this boundary will mix NULL (unknown) with 0.0 (zero actual) with valid values. The read path must explicitly exclude NULL rows from actuals sums (not treat them as zero), and should surface a "partial coverage" flag on rollups where some work items have no actuals.

6.5 Razor-semantic propagation to UI copy

Changing the razor type changes what the numbers mean: - `elapsed_total` → measures throughput / lead time; variance = "how long vs how long we thought" - `logged_effort` → measures resource consumption; variance = "how much effort vs how much we estimated" - `elapsed_active` → measures cycle time excluding wait; variance = "active work duration vs estimate"

These are different production signals and should not share generic "variance" labels in the UI. The read path must return `razor_type` alongside every actuals value so the frontend can render razor-appropriate labels. UI copy for each razor type needs to be defined (deferred to UX phase). The plan should not bake in "estimate accuracy" as the universal framing, it is only correct for effort razors.

7. Sequenced Milestones

M0, Razor config infrastructure (1-2 days, Tier 0 foundation) - Migration M-01:

`actuals_razor_config` table - API endpoint: `GET/POST /api/actuals/config` (studio reads and writes their razor) - Config validation: check that declared `source_field_id` exists in the studio's `source_field_mappings` - No sync changes yet; this is config-only - Reviewable: migration + two route handlers + Pydantic model

M1, Actual value extraction in sync pipeline (1-2 days, Tier 0) - Migration M-02:

`replicated_work.actual_value` - Normalizer change: `normalize_work()` accepts `razor_config`, extracts value from field - Writer change: write `actual_value` when present - Runner change: load razor config per studio at sync start, pass to `normalize_work` - Reviewable: diff on `normalizer.py` + `writer.py` + `runner.py`, verifiable with a manual sync

M2, Actuals read path + commensurability guardrail (1-2 days, Tier 0 complete) -

`routes/actuals.py`: `/rollup` and `/asset/{id}` endpoints - Commensurability check: unit comparison, `conversion_to_days` path, null-variance response - NumberBot context extension - Reviewable: routes + guardrail unit tests (commensurability logic should be isolated and testable without DB)

M3, canonical_work layer (1 day, Tier 1 foundation) - Migrations M-03 and M-04 (expand phase only, no NOT NULL yet) - `upsert_canonical_work` in `writer.py` - Backfill script (M-05) for testing on dev environment - Reviewable: migrations + writer diff

M4, Changelog capture (2-3 days, Tier 1) - `lib/sync/changelog.py` - Runner integration: pre-fetch current values, call changelog after upsert - Migration M-06 - Reviewable: changelog.py unit tests (diff logic isolated), integration test confirming events are written on a hash change

M5, Reconciliation (2-3 days, Tier 1) - `lib/sync/reconciliation.py` - Migration M-07 - Schedule.py: trigger reconciliation pass from `/reconcile-work` - Actuals route extension: per-asset reconciliation-linked detail - Reviewable: reconciliation unit tests for source_id and name-fuzzy match paths

M6, Tier 1 hardening + verification gate (1 day) - Run M-05 backfill on staging with production data snapshot - Confirm verification queries return 0 - Confirm work_changelog populated after a sync - Document the disappearance policy (canonical_work.deleted_at decision) - After confirmation: migration M-08 (NOT NULL constraint, IRREVERSIBLE) - Reviewable: data verification report, migration file

M7, UI (deferred, not in this plan) - Actuals dashboard components - Per-asset estimate-vs-actual panel in Asset Viewer (WorkTab) - Razor config UI in Settings - Razor-aware label copy

Total backend estimate: M0-M6 $\approx$ 9-13 engineering days. Tier 0 (M0-M2) $\approx$ 4-6 days. Tier 1 (M3-M6) $\approx$ 5-7 days. Each milestone is independently reviewable and deployable.